

Professores:

Celso Rodrigo Giusti (The Best)

Daniel Manoel Filho

Felipe Augusto Stevanim Gregate

Marlon Palata Fanger Rodrigues

REACT • AULA 1

Introdução e Primeiro Projeto



O que faremos hoje

- ✓ O que já fizemos: HTML, CSS, JS, DOM, OO.
- ✓ Aulas 0, 1 e 2: Bootstrap e Tailwind via CDN.
- Hoje: instalar React com Vite, estrutura do projeto e criar os primeiros componentes.
- Próximas aulas: useState, eventos, consumo de API.
- 🎯 **Fio condutor:** refatorar o TechFood em React.

REACT • O que é

O que é React?



React é uma **biblioteca JavaScript** para construir interfaces.

Criada e mantida pela **Meta** + comunidade open source.

A tela é montada com **componentes**: peças reutilizáveis.

Você cria uma peça e a **ENCAIXA** onde quiser, quantas vezes precisar.

Analogia: componentes são peças de Lego que você reaproveita.

● **Mercado — Meta, Airbnb, Netflix, iFood, Uber.**

Domina as vagas Front-End. Base para Next.js e React Native.

Por que React? O problema que ele resolve

Com JS puro, criar elementos é trabalhoso:

- `const el = document.createElement("article")`
- `el.innerHTML = `<h5>${prato.nome}</h5>``
- `container.appendChild(el)` ← repetir para cada item

Com React, você descreve a tela **uma vez**:

- `cardapio.map(p => <CardPrato key={p.id} nome={p.nome} />)`

E o React **atualiza a tela sozinho** quando os dados mudam.

Node, npm, npx — papéis de cada um

Confirme no terminal: `node -v` e `npm -v`

- Node.js — motor JS fora do navegador. React precisa dele.
- npm — gerencia dependências. `package.json` lista o que o projeto precisa. `npm install` baixa tudo.
- npx — executa pacotes sem instalar globalmente.
`npx create-vite@latest` → cria o projeto React.
- **Vite — build ultrarrápido.** Substitui o CRA (depreciado 2023).

O que é VITE?

- Ferramenta que **cria, roda e empacota** o projeto front-end.
- Criado por Evan You (o criador do Vue), 2020. "Vite" = rápido em francês.
- **Por que é rápido:** usa ES Modules do navegador + esbuild (feito em Go). HMR = salvou o arquivo, a tela atualiza na hora.

Concorrentes:

- Create React App (CRA) — era o padrão; depreciado em 2023.
- Webpack — clássico e poderoso, mas complexo e lento no dev.
- Parcel · esbuild (o Vite usa por dentro) · Turbopack.

🎯 **Por que usar hoje:** zero-config, start instantâneo e é o padrão recomendado pela doc do React (depois do fim do CRA).

Criando o projeto React — ao vivo

1. Criar o projeto:

```
npx create-vite@latest techfood-react -- --template react
```

2. Entrar na pasta e instalar:

```
cd techfood-react → npm install
```

3. Iniciar o servidor:

```
npm run dev → abre em localhost:5173
```

Salvou o arquivo? O **Vite atualiza a tela na hora** (HMR).

Estrutura de pastas — Vite + React

techfood-react/

```

├── src/
│   ├── components/  ← seus componentes
│   ├── App.jsx      ← componente raiz
│   ├── App.css
│   └── main.jsx      ← ponto de entrada
├── index.html        ← só tem <div id="root">
└── package.json      ← dependências + scripts
  
```

Limpendo o projeto padrão

App.jsx → apagar o conteúdo, deixar só a função vazia.

App.css → apagar todo o CSS padrão.

O HMR do Vite atualiza localhost:5173 instantaneamente.

Qualquer alteração salva → tela atualiza em milissegundos.

Agora o projeto está **limpo** para montar nossos componentes.

JSX — HTML dentro do JavaScript

JSX não é HTML. É JS com sintaxe parecida com HTML.

O **Vite** converte **JSX** em JavaScript puro antes de rodar.

HTML: `class="card"` → JSX: `className="card"`

HTML: `"texto fixo"` → JSX: `{prato.nome}`

HTML: `onclick="fn()"` → JSX: `onClick={fn}`

HTML: `<br>` → JSX: `<br />`

⚠ **Erros comuns:** esquecer `className`, não fechar tags.

Componente funcional — a peça básica

Função JS com nome em Maiúscula que retorna JSX.

```
function Header() {  
  return (  
    <header className="header">  
  
    </header>  
  )  
}
```

⚠ Nome em **Maiúscula é obrigatório** — diferencia de tags HTML.

Props — dados que vêm de fora

Props são os parâmetros do componente. **Pai passa** → **filho usa**.

```
// App.jsx — PAI passa os dados
```

```
<CardPrato nome="Feijoada" preco={42.90} />
```

```
// CardPrato.jsx — FILHO recebe e usa
```

```
function CardPrato({ nome, preco }) { ... }
```

⚠ **String: nome="valor"** Número/JS: preco={42.90}

ReactDOM.createRoot — como o React entra na página

O index.html tem só `<div id="root">`. O React gera tudo.

```
// main.jsx — ponto de entrada
```

```
import ReactDOM from 'react-dom/client'
```

```
import App from './App.jsx'
```

```
ReactDOM.createRoot(document.getElementById('root'))
```

```
.render(<React.StrictMode><App /></React.StrictMode>)
```

Live Code — TechFood em React



O que vamos fazer:

1. Criar Header.jsx — componente sem props
2. Criar CardPrato.jsx — props: nome, preco, categoria
3. Array cardapio no App.jsx — mesmo cardápio do TechFood
4. `.map()` → um `<CardPrato />` para cada prato com key

Ver **Guia de Live Coding** para o passo a passo completo.

Componentes no mundo real

Em dupla — 10 min:

- Olhem a tela do TechFood: quais partes se repetem?
- Qual delas viraria um componente reutilizável?
- O que muda de um card para outro? (isso vira props)

Pensar em componentes é começar a pensar como dev React.

Exercícios — TechFood React

Continue seu projeto (ou abra a pasta exercicios/).

● **NÍVEL 1 (todos):**

1. Adicione +2 pratos no array — veja o `.map()` criar os cards sozinho.
2. Crie o componente Rodape.jsx (`<footer>`) e use no App.

● **NÍVEL 2 (consolidação):**

3. Adicione a prop "descricao" no CardPrato e mostre no card.
4. Personalize a tagline do Header.

● **DESAFIO (quem terminar):**

5. Mostre 🍰 antes do nome quando a categoria for "Sobremesa".
6. Mostre "Cardápio com N itens" usando `cardapio.length`.

Resumo

- ✓ React: biblioteca JS baseada em componentes reutilizáveis.
- ✓ Vite + npm: ambiente moderno. `npm run dev` inicia o servidor.
- ✓ JSX: HTML dentro do JS. Use `className` no lugar de `class`.
- ✓ Componente: função Maiúscula + `export default`.
- ✓ Props: dados do pai para o filho — fluxo sempre top-down.
- ✓ `.map()`: gera uma lista de componentes, cada um com `key`.

Referências

REACT TEAM. React Documentation. Disponível em: react.dev. Acesso em: ago. 2026.

VITE TEAM. Vite Documentation. Disponível em: vitejs.dev. Acesso em: ago. 2026.

MOZILLA. MDN Web Docs — JavaScript. Disponível em: developer.mozilla.org/pt-BR. Acesso em: ago. 2026.

ALVES, W. P. Programação orientada a objetos com Java 8. São Paulo: SENAI-SP Editora, 2017.



ESCOLA SENAI ÍTALO BOLOGNA

Av. Goiás, 139

Telefone

(11) 2396-1999

Instagram

@senai.itu

Facebook

/senaisp.itu

Site

<https://sp.senai.br/unidade/itu/>